

AGENCYOS

WhatsApp Automation & Communication System

Document 08 of the AgencyOS Master Documentation Set

LEAD WHATSAPP • CLIENT PROJECT GROUP • INTERNAL AGENCYOS GROUP • MESSAGES •
VOICE NOTES • LANGUAGE DETECTION • APPROVALS • NOTIFICATIONS

Purpose: Define WhatsApp as the primary communication and operational channel connecting leads, clients, Admin and AI agents while keeping AgencyOS as the structured source of truth.

1. Core Communication Principle

WhatsApp is the human communication layer; AgencyOS is the operational source of truth.

- Leads can enter through WhatsApp.
- Sales Agent can communicate with leads through WhatsApp.
- Converted clients can be connected to a project-specific WhatsApp group.
- AI agents can communicate through approved channels.
- Admin can operate through an internal AgencyOS/Admin WhatsApp group.
- Important approvals, payments and decisions become structured AgencyOS records.
- Raw messages remain linked as evidence where permitted.
- No AI agent should depend on WhatsApp history alone to understand a project.

2. WhatsApp Channel Architecture

Channel	Participants	Purpose
Channel	Participants	Primary purpose
Lead WhatsApp	Lead + Sales Agent/agency	Qualification, trust building, requirements, quotation, follow-up
Client Project Group	Client + PM/agents/agency	Project communication, feedback, approvals, progress
Internal AgencyOS Group	Admin + authorized agents	Approvals, escalations, alerts, operational coordination
Optional Support Group	Client + Support/CS	Post-launch support and maintenance

3. Lead WhatsApp Workflow

NEW MESSAGE → IDENTIFY/CREATE LEAD → LANGUAGE DETECTION → SALES AGENT → QUALIFY → REQUIREMENTS → TRUST/OBJECTION → QUOTATION → ACCEPTANCE → PAYMENT/EXCEPTION → CONVERSION

- Create a lead when an unknown number contacts the configured business WhatsApp.
- Match existing client/lead by phone/identity where safe.
- Assign Sales Agent according to routing rules.
- Load relevant sales knowledge and conversation summary.
- Reply in the detected/preferred language.
- Extract requirements into structured lead memory.
- Track trust concerns.
- Prepare quotation within policy.
- Request Admin approval when required.
- Track acceptance and next action.
- On conversion, link the lead to the client/project.

4. Client Project WhatsApp Group

- Create/link a group after project onboarding according to workflow policy.
- Store group ID and project ID mapping.
- Add authorized participants only.
- PM Agent is the primary operational coordinator.

- Agents post only when their role requires communication.
- Client feedback is captured and linked to the project.
- Approvals reference exact versions/artifacts.
- Project milestones and safe status updates can be automated.
- Internal reasoning, credentials and private agency notes must never be posted to the client group.

5. Internal AgencyOS WhatsApp Group

This group is the operational control channel for Admin notifications, approvals and escalations.

- New approval requests.
- Payment verification requests.
- High-value quotations.
- Discount exceptions.
- Project blockers.
- Agent failures.
- AI provider outages.
- Deployment approval.
- Handover readiness.
- Security/critical alerts.
- Admin commands where the configured interaction supports them.

Every actionable message must carry a structured reference ID so the backend can validate and audit the action.

6. Message Processing Pipeline

INCOMING MESSAGE → WEBHOOK/EVENT → AUTHENTICATE SOURCE → MAP CHANNEL → IDENTIFY CLIENT/PROJECT → PARSE CONTENT → LANGUAGE → INTENT → POLICY → AGENT → RESPONSE → SEND → STORE EVENT

- Validate webhook/signature and provider event identity.
- Deduplicate incoming events.
- Map phone/group to organization, client, lead or project.
- Classify text/media/voice/document.
- Detect language.
- Detect intent.
- Retrieve relevant context.
- Select agent/model.
- Generate response.
- Apply safety/policy checks.
- Send message.
- Persist sent-message event and provider reference.

7. Idempotency & Duplicate Protection

- Store provider message/event IDs.
- Reject duplicate webhook events safely.
- Use idempotency keys for outbound side effects where supported.

- Do not send duplicate invoice/payment/approval notifications after retries.
- A timeout must not automatically imply that a message was not delivered.
- Track SENT, DELIVERED, READ, FAILED states where the provider exposes them.

Communication retries must be safe. Financial or approval side effects must be separated from message delivery.

8. Language Detection

- Detect the language of each incoming message where practical.
- Maintain client preferred language.
- Allow Admin/client to override detected language.
- Support mixed-language messages such as Hinglish.
- Do not translate technical names, product names or client-provided identifiers unnecessarily.
- Use language preference in Sales, Support and Customer Success prompts.
- Keep original message unchanged as source evidence.
- Store detected language/confidence where useful.

Language detection should improve communication, not change the underlying business meaning.

9. Voice Notes

VOICE NOTE → MEDIA REFERENCE → TRANSCRIPTION → LANGUAGE DETECTION → INTENT/REQUIREMENT EXTRACTION → CONTEXT UPDATE → RESPONSE

- Store the original media reference where permitted.
- Transcribe voice note.
- Detect language after transcription.
- Preserve the original transcript and link it to the media.
- Extract requirements, decisions, objections or approvals only when evidence supports them.
- If transcription is uncertain, do not silently invent words.
- Ask for clarification when a voice note is materially ambiguous.
- Client-facing response may be text or configured audio depending on the system design.

10. Voice Note Memory

- Message ID.
- Media reference.
- Transcript.
- Detected language.
- Timestamp.
- Speaker.
- Confidence/status.
- Extracted structured facts.
- Project/lead/client link.

The transcript is useful context, but exact approval/financial states should still require structured confirmation according to policy.

11. Message Types

- Text.
- Voice note.
- Image.
- Document/PDF.
- Video.
- Location where supported and appropriate.
- Reaction/read status where supported.
- Interactive approval/confirmation messages where supported.
- Template messages where provider policy requires them.

12. Intent Detection

Lead intents

- New enquiry.
- Service inquiry.
- Price inquiry.
- Requirement sharing.
- Trust concern.
- Negotiation.
- Quotation request.
- Acceptance.
- Follow-up.
- Not interested.

Client/project intents

- Progress inquiry.
- Feedback.
- Approval.
- Change request.
- Bug report.
- Payment message.
- Payment proof.
- Support request.
- Cancellation request.
- Handover request.
- New project inquiry.
- Upsell response.

13. Approval Messages

An approval message must identify exactly what is being approved.

- Approval ID.
- Client/project.
- Action.
- Amount where applicable.

- Artifact/version where applicable.
- Relevant summary.
- Approve / Reject / Request Changes.
- Expiry if applicable.
- Admin identity/authentication.

The backend validates that the approval is still valid before executing it. A later change to scope, quote, build or policy may invalidate the approval.

14. Client Approval Workflow

AGENT PREPARES ARTIFACT → CLIENT REVIEW REQUEST → CLIENT RESPONDS →
CONFIRM EXACT VERSION → STRUCTURED APPROVAL → UPDATE PROJECT STATE

- For UI, store exact design version.
- For prototype, store exact build/version.
- For scope, store exact scope version.
- For final acceptance, store exact release/handover version.
- Ambiguous responses should trigger a confirmation request.
- Do not infer acceptance from a generic 'looks good' when the policy requires explicit approval.

15. Payment Proof via WhatsApp

CLIENT SENDS PAYMENT PROOF → STORE MEDIA/REFERENCE → EXTRACT CLAIMED
AMOUNT/REFERENCE → LINK INVOICE/MILESTONE → PENDING_VERIFICATION →
ADMIN/FINANCE → VERIFIED / REJECTED / NEEDS_INFO

- Never mark payment VERIFIED solely because an image or message was received.
- Payment state is controlled by Finance/Policy workflow.
- Admin can inspect proof before verification.
- Verified payment unlocks only the workflow transitions permitted by policy.

16. Sales Trust Workflow

- Detect trust concern from conversation.
- Load approved trust-building responses.
- Share approved portfolio/samples/demos.
- Offer approved low-advance structure where eligible.
- Offer approved UI/prototype validation structure where eligible.
- Offer no-advance start only if enabled by policy.
- Request Admin approval for exceptions.
- Record exact terms accepted by client.

Sales Agent must never improvise a commercial exception outside the Policy Engine.

17. Automated Follow-Up

- Track last client interaction.
- Determine next follow-up according to sales policy.
- Avoid excessive messaging.
- Respect client opt-out/preferences.

- Use different follow-up templates based on lead stage.
- Stop follow-up when lead is converted, lost, opted out or otherwise closed.
- Escalate high-value leads or unusual objections when configured.

18. Project Notifications

- Project started.
- Payment verified.
- UI ready for review.
- Prototype ready for review.
- Development milestone ready.
- QA completed.
- Action required from client.
- Payment due.
- Payment overdue.
- Change request received.
- Project blocked.
- Final acceptance requested.
- Handover ready.
- Maintenance available.
- Support ticket update.

Notification rules must be policy-configurable and should avoid unnecessary client messages.

19. Agent Communication Rules

- Sales Agent communicates sales/lead matters.
- PM Agent communicates project coordination/status.
- UI/Prototype agents communicate only when configured; normally PM presents deliverables.
- Developer agents should not overwhelm clients with technical internal details.
- QA Agent can provide structured QA status through PM.
- Finance Agent communicates invoices/payment status through approved templates.
- Support Agent handles support requests.
- Customer Success handles post-launch relationship.
- Upsell Agent communicates only within commercial policy.
- Orchestrator should not expose internal routing details to clients.

20. Internal vs Client Visibility

Information	Client	Internal
Information	Client Group	Internal Group
Project progress	Yes	Yes
Client feedback	Yes	Yes
Payment request/status	Yes, appropriate details	Yes
Internal AI reasoning	No	Controlled
Agent failure diagnostics	No	Yes

API keys/secrets	Never	Never in chat
Private pricing strategy	No	Controlled
Admin policy details	No	Controlled
Approval request	Only relevant client approvals	Yes

21. Message Templates

- Welcome/new lead.
- Requirement clarification.
- Portfolio/sample sharing.
- Quotation.
- Trust concern.
- Payment request.
- Payment received/pending verification.
- UI review.
- Prototype review.
- Milestone completion.
- QA/update.
- Final acceptance.
- Handover.
- Maintenance renewal.
- Support acknowledgment.
- Upsell/repeat-project outreach.
- Templates should support variables, language variants, approval requirements and versioning.

22. AI Response Generation

- Retrieve relevant context first.
- Use agent-specific system instructions.
- Use client language preference.
- Apply business policy.
- Avoid unsupported promises.
- Avoid exposing internal details.
- Validate structured actions before sending.
- For high-risk messages, require approval.
- Record provider/model used where applicable.

The AI provider is replaceable; the WhatsApp communication layer should not depend on one model.

23. Human Handoff

If the AI cannot safely continue, it must escalate rather than fabricate.

- Client explicitly requests human.
- High-value negotiation.
- Legal/compliance concern.
- Sensitive complaint.

- Repeated misunderstanding.
- Payment dispute.
- Refund/cancellation dispute.
- Security issue.
- Repeated AI failure.
- Out-of-policy request.

Escalation should create a structured task for Admin/Sales/PM/Support with the relevant conversation context.

24. Message Queue & Delivery

- Outbound messages should pass through a queue.
- Support retries.
- Rate limiting.
- Provider error handling.
- Message priority.
- Deduplication.
- Delivery status updates.
- Dead-letter/failed-message queue.
- Admin visibility into stuck messages.
- Emergency pause.

A provider outage must not corrupt project state. Messages may queue or fail while the underlying project workflow remains intact.

25. WhatsApp Provider / Integration Layer

- Provider credentials stored in Admin API Keys/Integrations.
- Webhook endpoint.
- Webhook verification.
- Message send API.
- Media receive/download/reference handling.
- Delivery/read status handling where supported.
- Template management where required.
- Rate-limit handling.
- Error normalization.
- Provider health monitoring.
- Provider abstraction so the system can change WhatsApp integration providers if required.

Provider-specific implementation details should be isolated behind an internal AgencyOS communication adapter.

26. Security

- Validate incoming webhook authenticity.
- Never trust phone number alone for privileged Admin actions.
- Use authenticated Admin identity for high-risk approvals.
- Protect stored media and transcripts.
- Encrypt sensitive data at rest/in transit according to architecture.
- Do not expose secrets in messages.

- Restrict agent access by channel/project/client.
- Prevent cross-client message leakage.
- Audit privileged communication actions.
- Provide emergency messaging shutdown.

27. Tenant & Project Isolation

- Every message/event should carry organization context.
- Client messages map to one client/lead.
- Project groups map to one project.
- Agents cannot accidentally send a message to another client's group.
- Before outbound send, validate recipient/channel/project mapping.
- High-risk outbound actions should use a final destination check.

28. Communication Memory

- Raw message reference.
- Transcript.
- Message summary.
- Detected language.
- Intent.
- Client/project/lead linkage.
- Agent response.
- Provider/model.
- Delivery status.
- Approval reference where applicable.
- Related task.
- Related artifact.

Communication history feeds the AI Memory & Context System, while the original message remains the evidence source.

29. Notification Priority

- P0 — security, financial integrity, critical system outage.
- P1 — Admin approval, payment verification, production/deployment, blocked project.
- P2 — client action required, milestone updates, important support.
- P3 — routine progress, follow-up, non-critical notifications.

Priority determines notification channel, urgency and escalation policy.

30. Admin Controls

- Connect/disconnect WhatsApp integration.
- Configure numbers/accounts.
- View groups.
- Map group to project.
- Configure internal Admin group.
- Enable/disable AI messaging.

- Pause outbound messages globally.
- Pause one project.
- Pause one agent.
- Edit templates.
- Configure language defaults.
- Configure follow-up rules.
- Configure notification rules.
- View delivery failures.
- Retry safe messages.
- View communication audit.
- Test connection.

31. Conversation Search

- Search by phone.
- Client.
- Lead.
- Project.
- Group.
- Message ID.
- Date.
- Agent.
- Intent.
- Keyword.
- Approval ID.
- Payment reference.

Search must respect Admin permissions and client isolation.

32. Anti-Spam & Client Experience

- Rate-limit automated follow-ups.
- Respect opt-outs.
- Avoid duplicate notifications.
- Do not send multiple agents to the same client simultaneously unless coordinated.
- Use one primary conversational owner per workflow stage.
- Suppress routine messages when the client has already responded.
- Escalate instead of repeatedly repeating the same AI response.
- Maintain professional tone and language consistency.

33. Failure Handling

- Webhook failure → retry safely.
- Duplicate webhook → ignore after idempotency check.
- Transcription failure → ask client to repeat/type if important.
- Language detection uncertain → continue in known preference or ask.
- AI failure → fallback model/provider.

- Provider outage → queue/fallback/degraded mode.
- Message send failure → retry according to policy.
- Approval ambiguity → request explicit confirmation.
- Wrong group mapping detected → block send and alert Admin.
- Repeated failure → human/Admin escalation.

34. WhatsApp Approval Example

Admin receives: 'Approval Required — Project ABC — 20% milestone invoice. Amount █X. Invoice INV-123. Milestone: UI approved. Approve / Reject / Changes.'

- Admin selects the configured approval action.
- Backend authenticates the Admin and validates approval ID.
- Policy Engine confirms conditions remain valid.
- Approval is recorded.
- Finance/PM workflow continues.
- Audit record stores the decision.

The exact amount, project and invoice reference prevent a generic 'yes' from being attached to the wrong transaction.

35. WhatsApp Client Approval Example

PM sends UI Version 3 for review. Client confirms the exact version. AgencyOS records UI_VERSION_3_APPROVED and unlocks the configured next milestone.

- If the client later approves Version 4, Version 3 remains historical.
- If the client says 'make this change first,' the workflow becomes feedback/change-request, not automatic approval.

36. Conversation-to-Workflow Examples

New lead

WhatsApp → Sales → Qualification → Quote → Approval → Acceptance → Onboarding

Payment

WhatsApp proof → Finance → Verification → Milestone unlock

UI review

UI artifact → Client group → Feedback → Revision or approval → PM state update

Bug

Client message → Support → QA/Developer task → Fix → QA → Client update

New feature

Client request → Scope check → Change request → Quote/approval → Development

37. Acceptance Criteria

- Lead WhatsApp can create and update leads.
- Existing clients can be recognized safely.
- Client project groups map to projects.

- Internal AgencyOS/Admin group supports governed notifications/approvals.
- Messages are persisted and auditable.
- Webhook events are idempotent.
- Language detection works for configured languages and mixed-language messages.
- Voice notes can be transcribed and linked to source media.
- Important decisions become structured records.
- Payment proof is not automatically treated as verified payment.
- Approval messages reference exact versions/requests.
- AI can escalate to humans.
- Outbound messaging can be paused.
- Provider failures do not corrupt project state.
- Messages are isolated by tenant/client/project.
- Admin can manage integration credentials and communication rules.
- Templates are versioned.
- Delivery failures are visible.
- Communication history feeds the shared memory system.
- No agent can expose secrets/internal information to clients.
- Emergency messaging kill switch works.

38. Final Architecture

WHATSAPP → COMMUNICATION ADAPTER → EVENT/QUEUE → IDENTITY & CHANNEL
 MAPPING → MEMORY/CONTEXT → INTENT → AGENT → POLICY → AI MODEL →
 VALIDATION → WHATSAPP → AUDIT

WhatsApp becomes the natural conversational interface for the agency: leads can talk to Sales, clients can work with the Project Manager, Admin can approve governed actions through the internal group, and agents can coordinate operationally — while AgencyOS keeps the structured state, approvals, payments, project records and audit trail reliable.

The goal is simple: the client should experience one responsive agency, even though behind WhatsApp there is a coordinated workforce of specialized AI agents, Admin controls and multiple AI providers.